

Simulation et Contrôle d'un Mini-Robot Autonome

UL2IN013 - LesQuatresBots

BOUKLACHI LILIA - TAKOUAT ANIS - BOUKERCHA YASMINE

Table des matières

1	Introduction	1
2	Mode d'emploi	1
2.1	Installation	1
2.2	Utilisation de Git	1
2.3	Lancement du programme	1
2.4	Fonctionnement	1
3	Schéma fonctionnel	2
4	Architecture du projet	2
4.1	Diagramme UML – Modèle du robot	2
4.2	Diagramme UML – Contrôle et Stratégies	3
5	Équations du mouvement	3
5.1	Vitesse du robot	3
5.2	Position	3
5.3	Rotation	4
6	Illustration	4
7	Améliorations possibles	4
8	Conclusion	4

1 Introduction

Ce projet a pour objectif de concevoir un mini-robot autonome capable de se déplacer dans un environnement en évitant des obstacles, tout en réalisant des trajectoires simples comme un carré.

Deux environnements sont considérés :

- une simulation 2D développée avec Pygame,
- une exécution sur un robot réel.

Le code est disponible sur GitHub :

- [Dépôt principal](#)
- [Branche anis-tak-patch1 : développement](#)

2 Mode d'emploi

2.1 Installation

- Télécharger le projet depuis GitHub (`git clone`)
- Ouvrir le projet avec **Visual Studio Code**
- Installer Python et les dépendances : `pip install pygame`

2.2 Utilisation de Git

- `git clone` : récupérer le projet
- `git pull` : mettre à jour
- `git add` : ajouter les fichiers
- `git commit` : sauvegarder les modifications
- `git push` : envoyer sur GitHub

2.3 Lancement du programme

Simulation : `python main.py` (Activer `main_simulation()` dans `main.py`)

Robot réel : Activer la fonction `main_real()` dans le fichier `main.py`.

2.4 Fonctionnement

Une fois lancé :

- le robot se déplace automatiquement,
- il exécute des stratégies (ex : tracer un carré) en affichant sa position , son angle et sa vitesse ,
- les obstacles sont affichés,
- les collisions sont détectées et arrêtent le robot.

Interface avec le robot réel :

Une API est utilisée pour communiquer avec le robot physique.

La classe `Robot2IN013` permet de contrôler les moteurs et les capteurs.

La classe `RobotAdapter` sert d'adaptateur entre la simulation et le robot réel. Elle permet d'utiliser les mêmes commandes (avancer, tourner, etc.) dans les deux cas.

3 Schéma fonctionnel

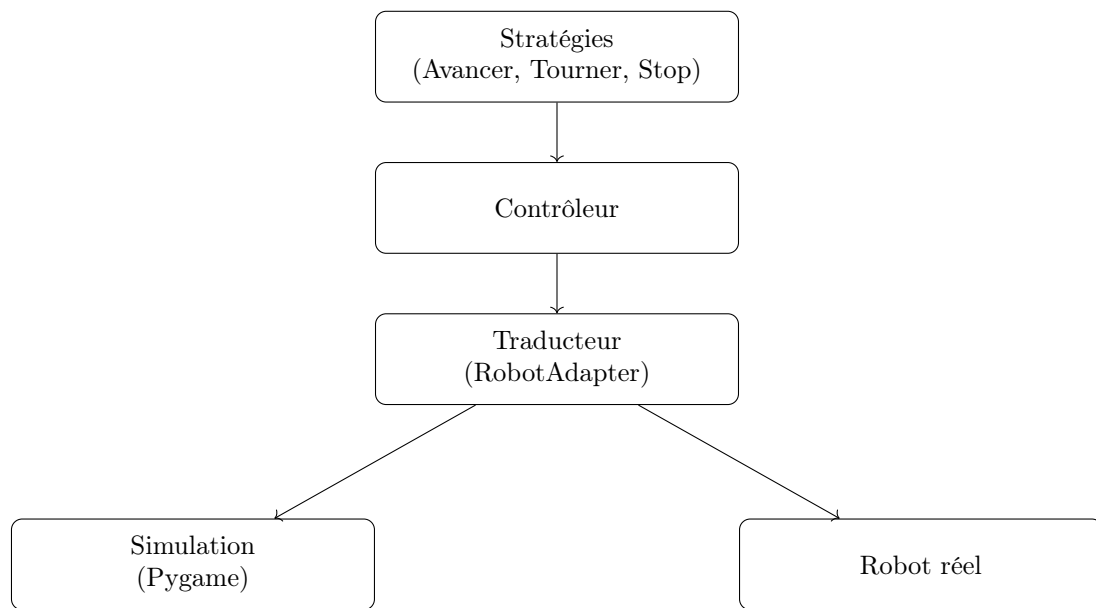


Figure 1 : Architecture complète du système .

4 Architecture du projet

4.1 Diagramme UML – Modèle du robot

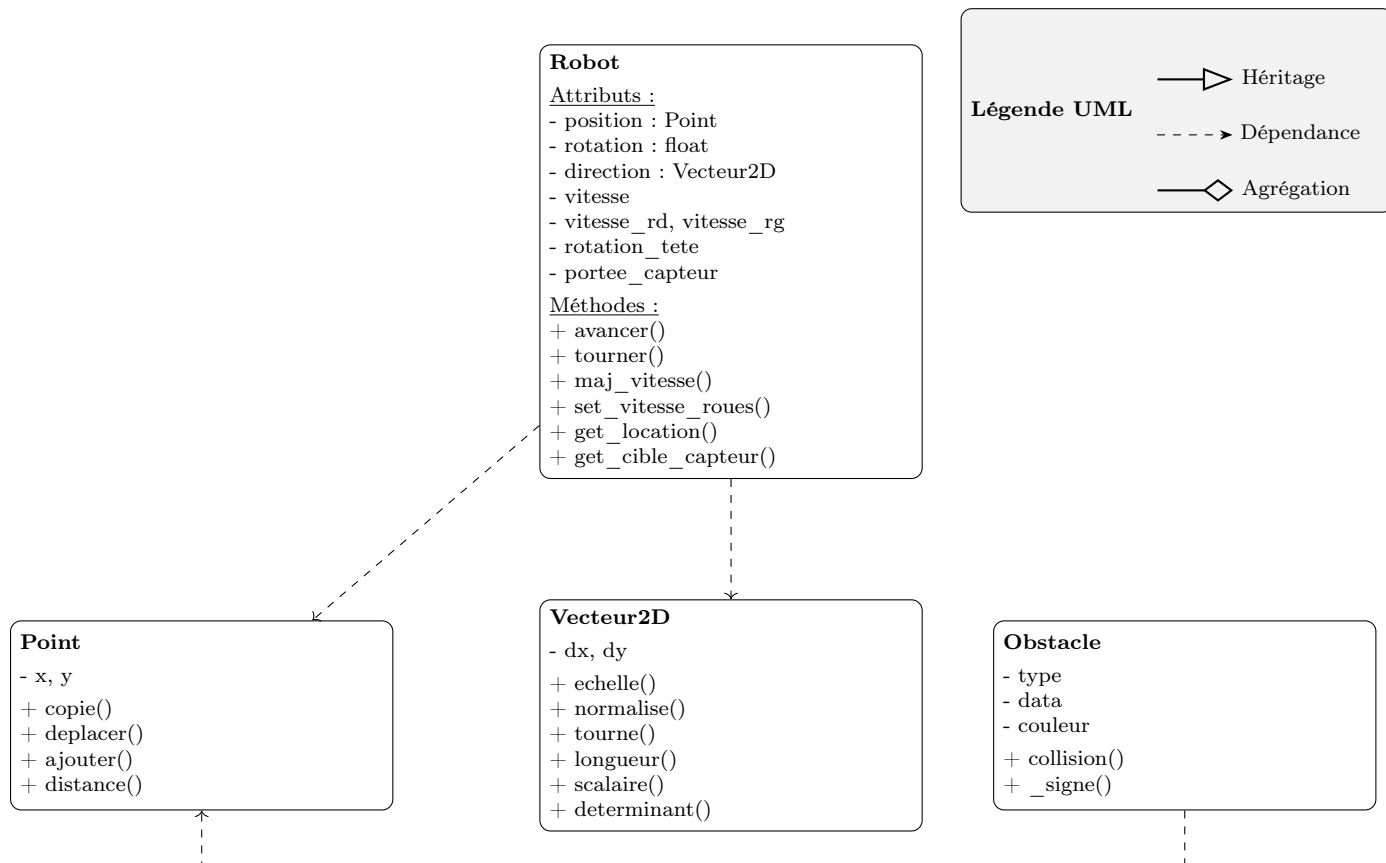


Figure 2 : Diagramme UML du modèle du robot.

4.2 Diagramme UML – Contrôle et Stratégies

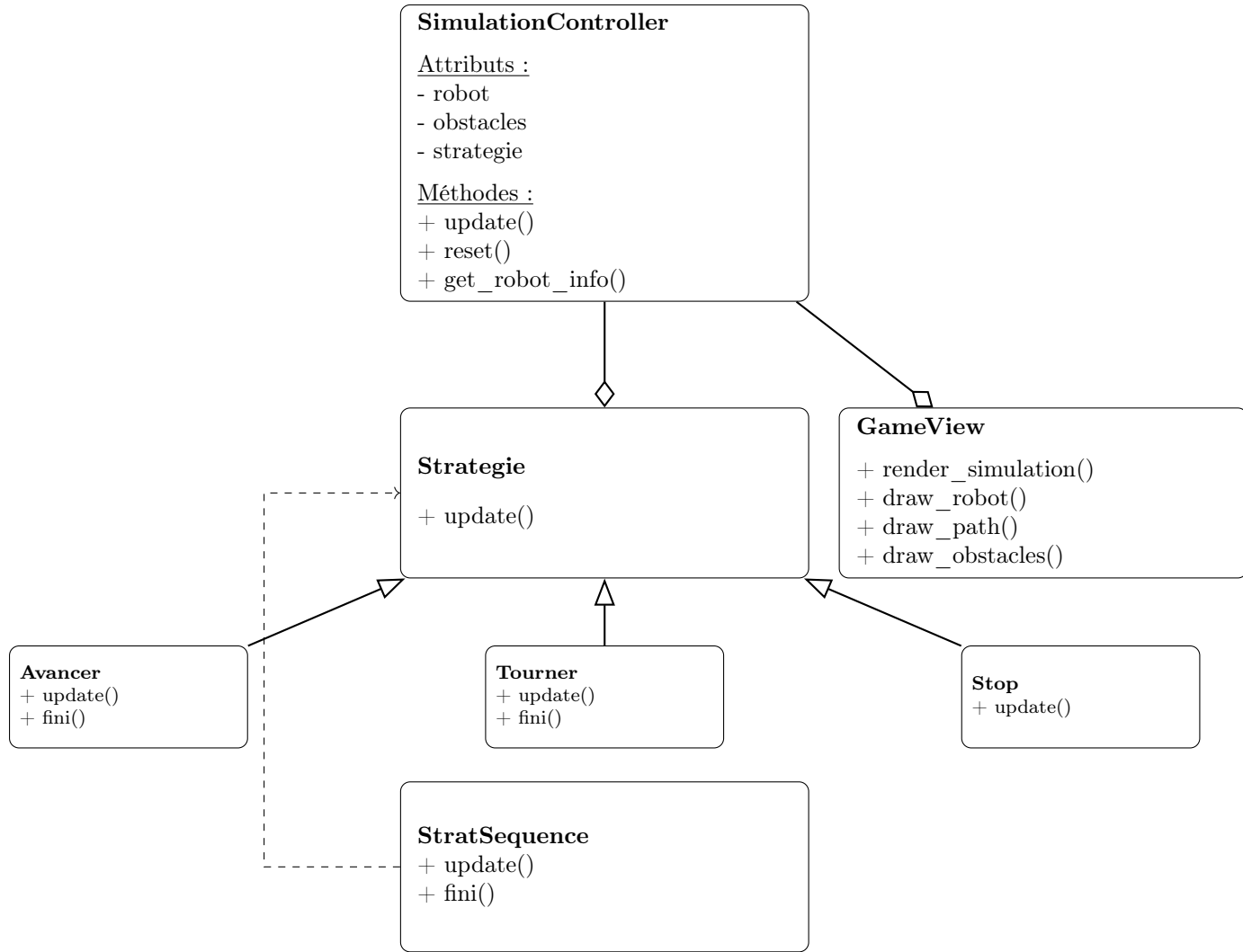


Figure 3 : Diagramme UML des composants de contrôle et des stratégies.

5 Équations du mouvement

Le robot est modélisé comme un robot différentiel. La vitesse linéaire n'est pas constante et dépend des vitesses des roues motrices.

5.1 Vitesse du robot

$$v(t) = \frac{v_d(t) + v_g(t)}{2} \quad (1)$$

5.2 Position

$$x_{t+1} = x_t + v(t) \cos(\theta_t) \Delta t \quad (2)$$

$$y_{t+1} = y_t + v(t) \sin(\theta_t) \Delta t \quad (3)$$

Ces équations décrivent le déplacement en fonction de la direction du robot et de la vitesse instantanée.

5.3 Rotation

$$\theta_{t+1} = \theta_t + \omega(t) \Delta t \quad (4)$$

avec :

$$\omega(t) = \frac{v_d(t) - v_g(t)}{L} \quad (5)$$

où :

- $v_d(t)$: vitesse de la roue droite,
- $v_g(t)$: vitesse de la roue gauche,
- L : distance entre les roues,
- Δt : pas de temps de simulation.

Ces équations permettent de simuler un comportement réaliste du robot différentiel avec vitesses variables.

6 Illustration

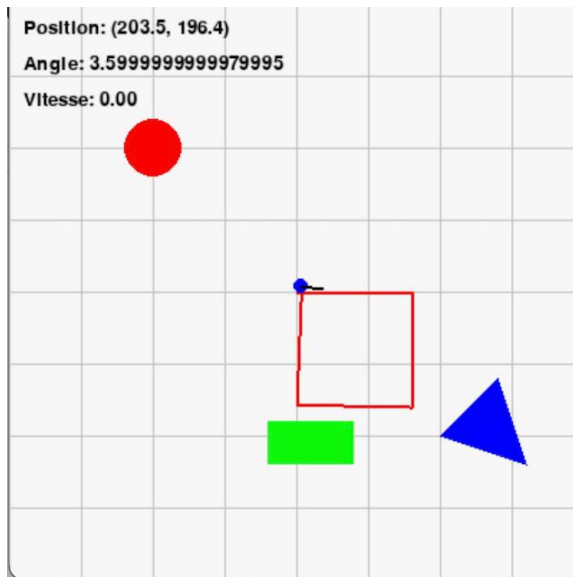


Figure 4 : Simulation du robot avec obstacles

7 Améliorations possibles

Plusieurs améliorations peuvent être apportées à ce projet afin de le rendre plus réaliste :

- Améliorer l'algorithme d'évitement d'obstacles avec des méthodes plus avancées .
- Rendre les capteurs et le modèle physique plus réalistes en intégrant du bruit, des erreurs de mesure, ainsi que des effets physiques comme l'inertie et les frottements.

8 Conclusion

Ce projet met en œuvre la modélisation du mouvement, l'interaction avec l'environnement et structuration logicielle . Il a été réalisé en équipe, avec une organisation des tâches via l'outil Trello. Il constitue une base solide pour des développements plus avancés.